

Database Model Checkpoint: From Current ERD to Target ERD

This presentation shows that the current database can be imported and that the planned future ERD includes the new customer requirements.

1. Working import

Show that the provided SQL script imports without syntax errors and that existing foreign keys are valid.

2. Current model

Explain the original tables: customer, supplier, ingredient, order and order ingredient.

3. Target model

Present the updated ERD with recipes, nutrition categories and restrictions.

Opening sentence

"At this checkpoint, we show that the current database import works and that our updated ERD is ready as the planned future database structure."

1. Checkpoint Requirements

What must be shown?

- The provided database script imports without syntax errors.
- The existing tables are correctly connected by foreign keys.
- The updated ERD shows the planned future database structure.

- The new customer entities and tables are visible.

What is not required yet?

- No complete SQL implementation of the extended ERD.
- No complex SQL queries yet.
- No stored procedures yet.

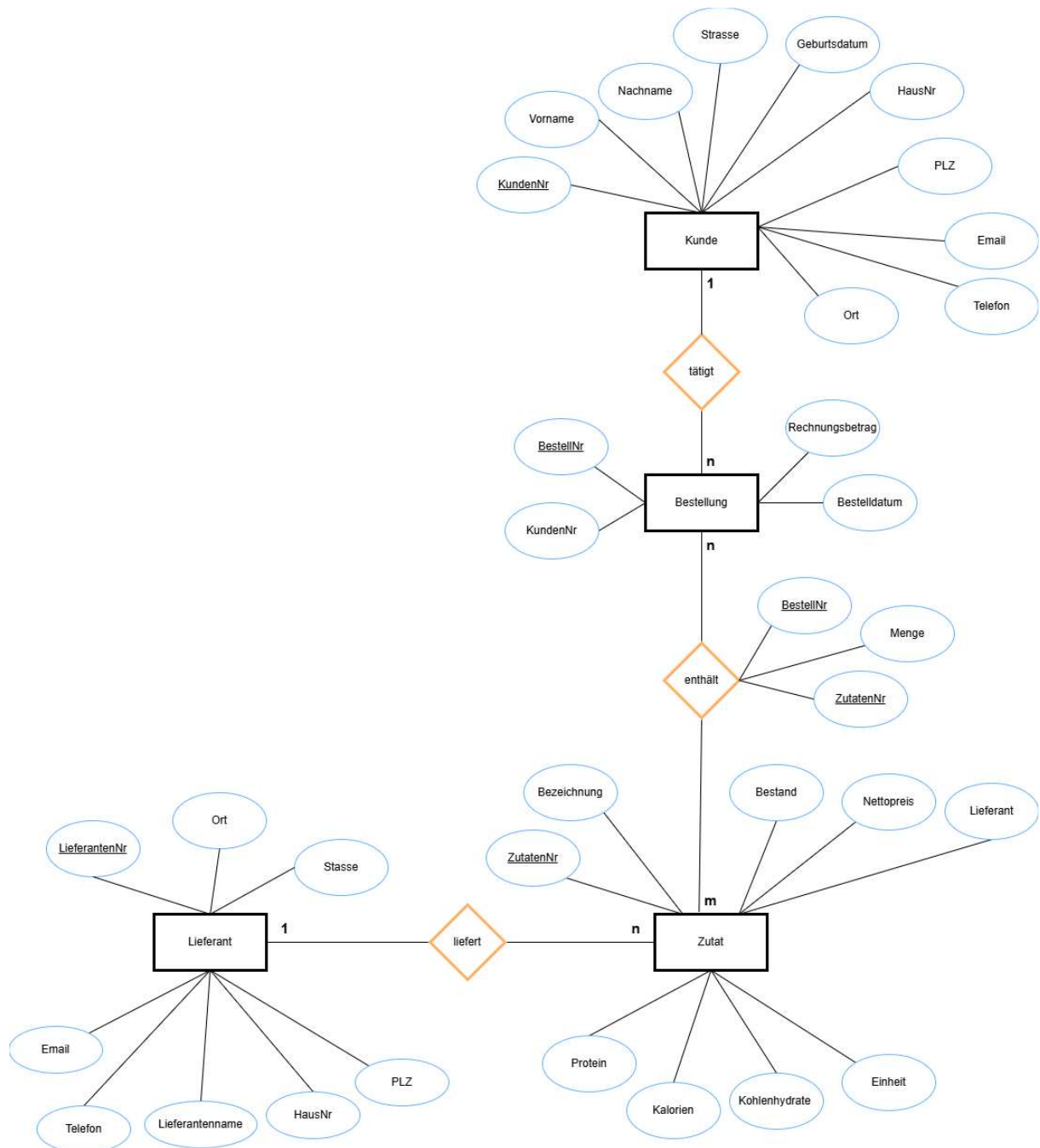
Speaker text

"The checkpoint is not the final implementation. Its purpose is to review the database model early, before we write complex queries."

2. Current ERD: Original Database

The original database already supports the basic order process.

Entity	Purpose	Key idea
KUNDE	Stores customer data.	One customer can place many orders.
LIEFERANT	Stores suppliers.	One supplier can deliver many ingredients.
ZUTAT	Stores ingredients, stock, price and basic nutrition values.	Ingredients are used in orders.
BESTELLUNG	Stores orders.	Each order belongs to one customer.
BESTELLUNGZUTAT	Connects orders and ingredients.	Resolves the existing many-to-many relationship.



Speaker text

"The original ERD contains customers, suppliers, ingredients, orders and a junction table for order ingredients. It can store orders, but it cannot store recipes yet."

3. Proof: Database Import Works

The script `Dump importieren.sql` creates the existing database tables and inserts test data.

Important foreign-key order

1. KUNDE
2. LIEFERANT
3. ZUTAT references LIEFERANT
4. BESTELLUNG references KUNDE
5. BESTELLUNGZUTAT references BESTELLUNG and ZUTAT

```
FOREIGN KEY (KUNDENNR) REFERENCES KUNDE (KUNDENNR)
FOREIGN KEY (LIEFERANT) REFERENCES LIEFERANT (LIEFERANTENNR)
FOREIGN KEY (BESTELLNR) REFERENCES BESTELLUNG (BESTELLNR)
FOREIGN KEY (ZUTATENNR) REFERENCES ZUTAT (ZUTATENNR)
```

Speaker text

"The import works because the tables are created and filled in the correct order.
The foreign key constraints connect customers to orders, suppliers to ingredients,
and orders to ingredients."

4. Customer Requirements

- Recipes consist of several ingredients.
- Nutrition categories must be stored.
- Restrictions or allergens must be stored.
- Ingredients must be assigned to recipes with an amount.
- Recipes must be selectable by nutrition category.

Kraut & Rueben Checkpoint Mid-project database model review, max. 15 minutes, English

Umzusetzende Kundenanforderungen:

- Speicherung von Rezepten bestehend aus mehreren Zutaten
- Speicherung von Ernährungskategorien
- Speicherung von Beschränkungen/ Allergenen
- Zusammenstellung von Zutaten entsprechend eines Rezepts
- Auswahl von Rezepten entsprechend vorgegebener Ernährungskategorien
- Auswahl bzw. Ausschluss von Rezepten auf Basis von Beschränkungen
- optional: Zugriffskontrolle personenbezogener Daten

Die Kraut und Rüben bittet Sie darüber hinaus ein Konzept zu entwickeln für die Fälle,

- dass ein Kunde Auskunft seiner Daten nach DSGVO beantragt oder
- die Löschung seiner Daten verlangt.

Klären Sie für beide Fälle sowohl die rechtlichen Grundlagen und implementieren Sie die technische Umsetzung (Beispielsweise als SQL-Skript).

Beispielhafte

Ernährungskategorien,

- Vegan
- Vegetarisch
- Frutarisch
- Low Carb
- High Protein
- ...

Beschränkungen

- Laktose
- Gluten
- Erdnuss
- Ei
- Tomate
- ...

und Nährstoffangaben

Nährstoffangaben	pro 100 g
Kalorien	2000kj/ 478kcal
Proteine	2.1g
Kohlenhydrate	77.2g
- davon Zucker	12.4g
Fett	17.9g
- davon gesättigte	13.9g
Fettsäuren	
Ballaststoffe	6.2g
Natrium	0.03g

Anforderungen an die von Ihnen zu entwickelnden Abfragen:

- Setzen Sie die folgenden Anforderungen mittels SQL-Abfragen um:
 - Auswahl aller Zutaten eines Rezeptes nach Rezeptname
 - Auswahl aller Rezepte einer bestimmten Ernährungskategorie
 - Auswahl aller Rezepte, die eine gewisse Zutat enthalten
 - Berechnung der durchschnittlichen Nährwerte aller Bestellungen eines Kunden
 - Auswahl aller Zutaten, die bisher keinem Rezept zugeordnet sind
- optional für FISI und ITSE bzw. verpflichtend für FIAE und FIDP:
 - Auswahl aller Rezepte, die eine bestimmte Kalorienmenge nicht überschreiten
 - Auswahl aller Rezepte, die weniger als fünf Zutaten enthalten
 - Auswahl aller Rezepte, die weniger als fünf Zutaten enthalten und eine bestimmte Ernährungskategorie erfüllen
- Erstellen Sie mindestens drei weitere Abfragen
- Stellen Sie sicher, dass Sie insgesamt mindestens je eine Abfrage mit **inner join**, **left join/ right join** sowie **Subselects** und **Aggregatfunktionen** erstellen
- Für FIAE/ FIDP: Setzen Sie mindestens drei Stored Procedures in Ihrer Datenbank um.

Speaker text

"From these requirements we derived that the old model has to be extended with recipes, nutrition categories, restrictions and additional nutrition values."

5. Missing Parts in the Current ERD

Missing area	Why it is missing	Planned extension
Recipes	No table for recipe name, portions or preparation.	REZEPT
Ingredients per recipe	BESTELLUNGZUTAT is for orders, not recipes.	REZEPT_ZUTAT

Nutrition categories	Categories such as vegan or low carb are not modeled.	ERNAEHRUNGSKATEGORIE
Recipe-category assignment	A recipe can belong to several categories.	REZEPT_ERNAEHRUNGSKATEGORIE
Allergens / restrictions	Allergens such as lactose, gluten or egg are missing.	BESCHRAENKUNG
Ingredient-restriction assignment	Restrictions come from ingredients.	ZUTAT_BESCHRAENKUNG

6. New Entities in the Target ERD

RECIPE / REZEPT

REZEPTNR
REZEPTNAME
PORTIONEN
ZUBEREITUNG

NUTRITION CATEGORY

KATEGORIENR
NAME
Examples: vegan, vegetarian, low carb, high protein.

RESTRICTION

BESCHRAENKUNGNR
NAME
Examples: lactose, gluten, peanut, egg, tomato.

Speaker text

"These are master entities. Storing them separately avoids repeated text values and makes the model easier to extend."

7. New Junction Tables

Relationship	Reason	Junction table
REZEPT n:m ZUTAT	A recipe has many ingredients, and an ingredient can be used in many recipes.	REZEPT_ZUTAT
REZEPT n:m ERNAEHRUNGSKATEGORIE	A recipe can belong to several categories.	REZEPT_ERNAEHRUNGSKATEGORIE
ZUTAT n:m BESCHRAENKUNG	An ingredient can have several allergens, and one allergen can appear in many ingredients.	ZUTAT_BESCHRAENKUNG

Speaker text

"Every many-to-many relationship is resolved with a junction table. This is important for a normalized relational database model."

8. Primary Keys and Foreign Keys

Primary key rule

A primary key uniquely identifies one row.

- REZEPTNR identifies one recipe.
- ZUTATENNR identifies one ingredient.

- KATEGORIENR identifies one category.

Foreign key rule

A foreign key points to the primary key of another table.

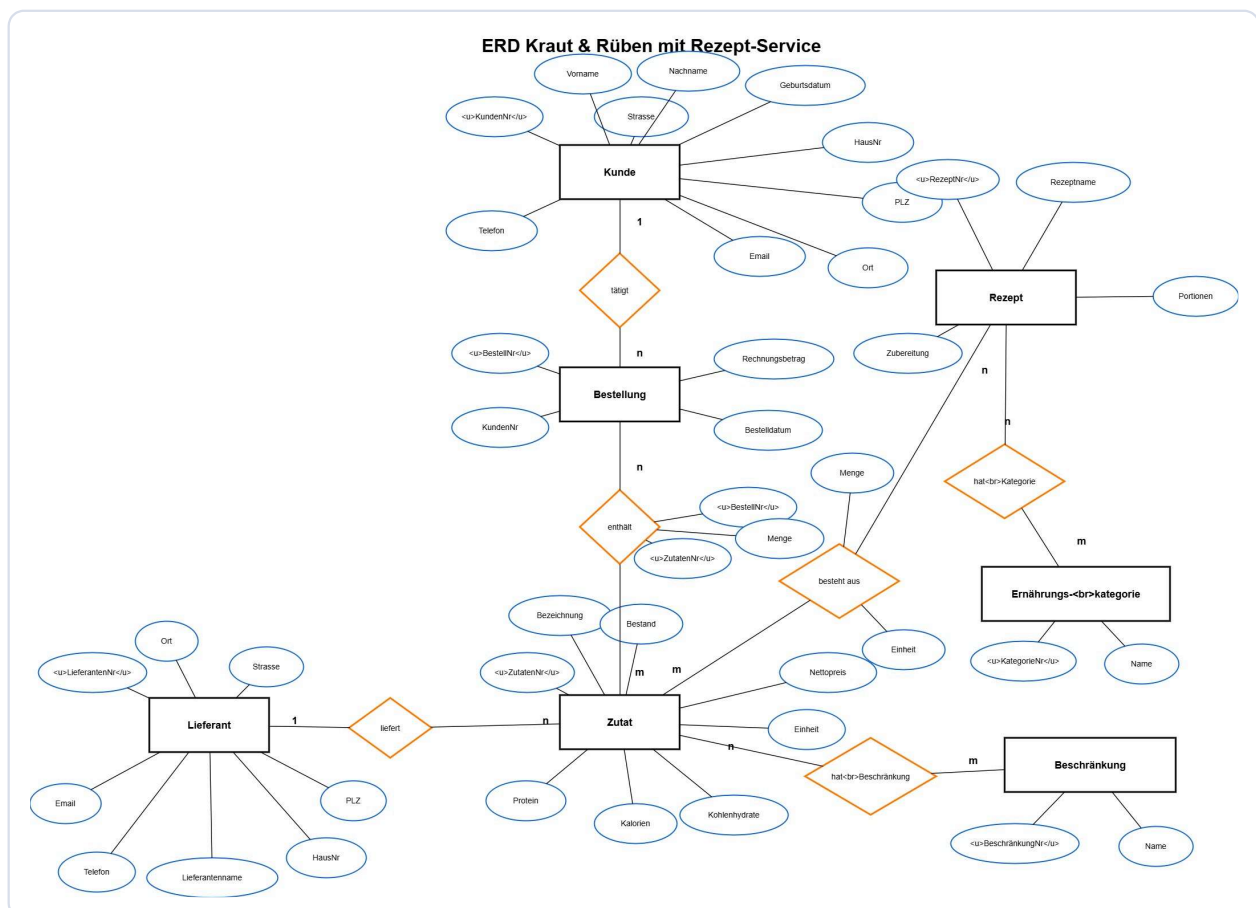
- REZEPT_ZUTAT.REZEPTNR points to REZEPT.
- REZEPT_ZUTAT.ZUTATENNR points to ZUTAT.

Speaker text

"In a junction table, both foreign keys usually form the combined primary key. This prevents duplicate assignments."

9. Updated Target ERD

The target ERD keeps the existing tables and adds the new entities and relationships required by the customer.



Speaker text

"This is the planned future structure. It includes Recipe, Nutrition Category and Restriction, plus the junction tables needed for the many-to-many relationships."

10. Normalization Check

1NF

No lists inside one column. Recipe ingredients are stored in REZEPT_ZUTAT.

2NF

MENGE and EINHEIT depend on the full key REZEPTNR + ZUTATENNR.

3NF

Categories and restrictions are stored once in their own tables.

Speaker text

"The target model is normalized because master data is stored once and many-to-many relationships are not stored as text lists."

11. Next Step After the Checkpoint

The full SQL implementation of the extended ERD is planned after the checkpoint. The ERD already shows how this will work:

- Each entity becomes a table.
- Each primary key becomes a PRIMARY KEY.
- Each relationship becomes a FOREIGN KEY.
- Each many-to-many relationship becomes a junction table.

```
CREATE TABLE REZEPT_ZUTAT (  
    REZEPTNR INTEGER NOT NULL,
```

```
ZUTATENNR INTEGER NOT NULL,  
MENGE DECIMAL(10,2),  
EINHEIT VARCHAR(25),  
PRIMARY KEY (REZEPTNR, ZUTATENNR),  
FOREIGN KEY (REZEPTNR) REFERENCES REZEPT(REZEPTNR),  
FOREIGN KEY (ZUTATENNR) REFERENCES ZUTAT(ZUTATENNR)  
);
```

Speaker text

"This SQL is the next implementation step, but it is derived directly from the ERD."

12. Final Conclusion

At this checkpoint, the current database import works and the foreign key constraints connect the existing tables correctly.

Based on the customer requirements, the target ERD adds Recipe, Nutrition Category and Restriction. The many-to-many relationships are resolved with junction tables.

Final sentence

"The next step is to implement this extended model in SQL and then write the required queries."

Files used: Dump importieren.sql, 91_Alt_IST_Zustand_ERD.png, 26ac7835-ca3c-4c56-ba0d-f39afa3ecd50 (2).png, 06_M-9_ERM_Diagramm_1.jpg